PROMETEO

thePower
FP Oficial

# APLICACIÓN DE ESCRITORIO PARA LA GESTIÓN DEL PRE-CHECK-IN HOTELERO

Grado Superior en Desarrollo de Aplicaciones Multiplataforma

Alumno: **Jon Golpe Villar**

Tutor del TFG: **Luz María Álvarez Moreno**

PROMETEO

DEDICATORIA (OPCIONAL)

# ÍNDICES

De contenido, tablas e ilustraciones. Se recomienda realizarlos de manera automática.

# ABSTRACT

[ESP]

Este proyecto consiste en el desarrollo de una aplicación de escritorio orientada a gestionar el pre-check-in de huéspedes en un hotel. La idea surge de mi experiencia trabajando en recepciones de hotel y viendo que los pequeños establecimientos no tienen herramientas sencillas para la recopilación de datos de huéspedes para el check-in.

La aplicación se ha desarrollado en Java, utilizando Swing para la interfaz gráfica y SQLite como base de datos. Permite gestionar huéspedes, habitaciones y estancias mediante operaciones básicas, todo desde una única interfaz. Además, incorpora algunas reglas de negocio importantes, como la validación de fechas y el control de disponibilidad de habitaciones en función de si existen solapamientos entre estancias.

A nivel interno, el proyecto se ha organizado siguiendo una estructura por capas (vista, servicio y acceso a datos), con el objetivo de mantener el código más ordenado y facilitar futuras mejoras.

El desarrollo se basa en una situación real del entorno hotelero, donde es necesario gestionar correctamente los datos de los huéspedes durante el proceso de check-in. Por ello, se ha tenido en cuenta la posibilidad de generar archivos XML compatibles con los requisitos oficiales de comunicación de viajeros para la policía.

En resumen, el resultado es una aplicación funcional que cubre los aspectos básicos del pre-check-in, y que puede servir como base para seguir evolucionando hacia una solución más completa.

[ENG]

This project involves the development of a desktop application designed to manage the pre-check-in process for hotel guests. The idea comes from my own experience working at hotel reception desks, where I noticed that small establishments often lack simple tools to collect guest data for check-in.

The application has been developed in Java, using Swing for the graphical interface and SQLite as the database system. It allows the management of guests, rooms, and stays through basic operations, all within a single interface. In addition, it includes important business rules, such as date validation and room availability control based on whether there are overlapping stays.

Internally, the project is structured using a layered approach (view, service, and data access), with the aim of keeping the code well organized and making future improvements easier.

The development is based on a real situation in the hotel environment, where it is necessary to properly manage guest data during the check-in process. For this reason, the possibility of generating XML files compatible with official traveler registration requirements for the police has been taken into account.

In summary, the result is a functional application that covers the basic aspects of pre-check-in and can serve as a foundation for further development into a more complete solution.

# JUSTIFICACIÓN DEL PROYECTO

La motivación principal de este proyecto surge de una necesidad real observada en el entorno hotelero: la gestión de los datos de los huéspedes durante el proceso de check-in. En muchos casos, esta gestión implica recoger una gran cantidad de información personal, validar datos y organizar estancias, lo que puede resultar complicado si se realiza mediante herramientas poco adaptadas o procesos manuales, sobre todo, en pequeños establecimientos.

El proyecto consiste en el desarrollo de una aplicación de escritorio orientada al pre-check-in de huéspedes. La elección de este enfoque responde a una experiencia profesional propia. En lugar de intentar desarrollar un sistema completo de gestión hotelera (PMS), que sería excesivamente complejo para el alcance de un Trabajo de Fin de Grado, he optado por centrarme en una parte concreta del proceso: la recogida y gestión de datos de los huéspedes antes de su llegada.

Este enfoque permite trabajar sobre un problema realista y técnicamente abordable, evitando caer en soluciones demasiado amplias. Además, facilita aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo, como la programación orientada a objetos, el diseño de interfaces gráficas, el acceso a bases de datos, es decir, los fundamentos más importantes de estos dos años de estudio.

Otro aspecto relevante es la relación del proyecto con requisitos reales del sector. La normativa de registro hotelero exige que los establecimientos recopilen y comuniquen información detallada de los huéspedes, incluyendo datos como identidad, fecha de nacimiento, nacionalidad, dirección y medios de contacto, entre otros. Esto implica que cualquier sistema que gestione el check-in debe estar preparado para manejar correctamente estos datos y validarlos.

En este sentido, el proyecto no solo tiene una finalidad académica, sino que también se inspira en un caso de uso real. La aplicación desarrollada puede servir para la generación de ficheros XML compatibles con sistemas oficiales de comunicación de viajeros.

En cuanto a aplicaciones similares, existen soluciones comerciales completas que gestionan todos los aspectos de un hotel (reservas, facturación, clientes, etc.). Sin embargo, estas herramientas suelen ser complejas, cerradas y fuera del alcance de un proyecto individual. La propuesta desarrollada en este trabajo no pretende competir con ellas, sino

ofrecer una solución simplificada, centrada en un problema concreto y diseñada desde cero como ejercicio práctico.

Finalmente, el público objetivo de esta aplicación serían pequeños alojamientos como hoteles boutique, pensiones o apartamentos turísticos, donde se necesite una herramienta sencilla para gestionar huéspedes y estancias sin la complejidad de un sistema completo. No obstante, su principal valor reside en demostrar la capacidad de diseñar e implementar una solución funcional, estructurada y ampliable.

# INTRODUCCIÓN

Este proyecto se centra en el desarrollo de una aplicación de escritorio orientada a facilitar el proceso de pre-check-in en un hotel, poniendo especial énfasis en la recogida de datos de los huéspedes antes de su llegada. El objetivo principal es que el establecimiento pueda disponer de toda la información necesaria con antelación, especialmente aquella que posteriormente debe ser comunicada a la policía según la normativa vigente.

La idea surge de la necesidad de contar con una herramienta sencilla que permita recopilar estos datos de forma organizada y anticipada. Esto resulta especialmente útil en hoteles pequeños, alojamientos turísticos o establecimientos que no disponen de recepción 24 horas, donde la llegada de los huéspedes puede producirse sin personal disponible en ese momento. En estos casos, disponer previamente de los datos facilita tanto la gestión interna como el cumplimiento de las obligaciones legales.

La aplicación permite gestionar huéspedes, habitaciones y estancias desde una única interfaz. El usuario puede dar de alta nuevos huéspedes, registrar habitaciones y crear estancias asociando personas, habitaciones y fechas. De esta forma, el establecimiento puede solicitar los datos al cliente antes de su llegada y tenerlos ya preparados en el sistema cuando se produzca el check-in.

Uno de los problemas más habituales en este tipo de entornos es la recogida de datos de forma manual o mediante herramientas poco adaptadas, lo que puede provocar errores, información incompleta o dificultades a la hora de cumplir con los requisitos legales. Para reducir estos problemas, la aplicación incorpora validaciones básicas, como comprobar que las fechas son correctas o que una habitación no está ocupada en el mismo rango de fechas. Además, la disponibilidad se calcula en función del solapamiento de estancias, lo que permite una gestión más realista.

A nivel interno, el proyecto se ha estructurado en capas (vista, servicio y acceso a datos), con el objetivo de mantener el código organizado y facilitar futuras mejoras. Aunque se trata de una aplicación sencilla, esta separación permite que la lógica de negocio esté desacoplada de la interfaz, lo que facilita su mantenimiento y evolución.

En cuanto a los requisitos principales del sistema, se pueden resumir en los siguientes puntos:

- ➔ Permitir el registro, consulta, modificación y eliminación de huéspedes.
- ➔ Gestionar habitaciones con sus características básicas.
- ➔ Crear estancias asociando huéspedes, habitaciones y fechas.
- ➔ Recopilar de forma anticipada los datos necesarios para el registro de viajeros.
- ➔ Validar que las fechas introducidas sean correctas (fecha de salida posterior a la de entrada).
- ➔ Comprobar la disponibilidad de habitaciones en función de las estancias ya registradas..
- ➔ Mantener los datos almacenados de forma persistente mediante una base de datos.
- ➔ Generación de archivos XML compatibles con la comunicación a la policía.

En conjunto, el proyecto busca ofrecer una solución sencilla pero práctica a un problema real del entorno hotelero, centrada en la recogida anticipada de datos y en facilitar tanto la gestión del establecimiento como el cumplimiento de la normativa.

# OBJETIVOS

Objetivo general:

Desarrollar una aplicación de escritorio que permita gestionar el pre-check-in de huéspedes en un alojamiento, facilitando la recogida anticipada de datos necesarios para el registro de viajeros y su posterior comunicación a la policía, mediante una interfaz sencilla y una estructura de datos organizada.

Objetivos específicos:

- ➔ Diseñar una base de datos para almacenar información de huéspedes, habitaciones y estancias.
- ➔ Implementar un sistema de gestión de huéspedes con operaciones básicas (alta, consulta, modificación y eliminación).
- ➔ Implementar la gestión de habitaciones y sus características principales.
- ➔ Desarrollar la gestión de estancias asociando huéspedes, habitaciones y fechas.
- ➔ Validar reglas básicas de negocio, como el control de fechas y la disponibilidad de habitaciones.
- ➔ Facilitar la recogida de datos de los huéspedes antes de su llegada.
- ➔ Preparar el sistema para la generación de archivos XML para comunicación a la policía.
- ➔ Organizar el proyecto mediante una arquitectura por capas para mejorar su mantenimiento.

**RFTP**

R01 – El sistema debe permitir la recogida y gestión de datos de huéspedes

R01F01 – Registrar huéspedes en el sistema

R01F01T01 – Diseñar la tabla huésped en la base de datos

R01F01T01P01 – Insertar registros de prueba y comprobar su almacenamiento

R01F01T02 – Crear formulario para introducir datos de huésped

R01F01T02P01 – Introducir un huésped desde la interfaz y verificar que se guarda correctamente

R01F01T03 – Validar campos obligatorios

R01F01T03P01 – Intentar guardar un huésped sin datos obligatorios y comprobar que se muestra error

R01F02 – Consultar y buscar huéspedes

R01F02T01 – Mostrar huéspedes en una tabla

R01F02T01P01 – Verificar que los datos se cargan correctamente en la interfaz

R01F02T02 – Implementar buscador de huéspedes

R01F02T02P01 – Filtrar resultados en función del texto introducido

R01F03 – Modificar y eliminar huéspedes

R01F03T01 – Permitir editar datos de un huésped

R01F03T01P01 – Modificar un registro y comprobar que se actualiza en la base de datos

R01F03T02 – Permitir eliminar huéspedes

R01F03T02P01 – Eliminar un huésped y verificar que desaparece de la tabla

R02 – El sistema debe permitir gestionar habitaciones

R02F01 – Registrar habitaciones

R02F01T01 – Diseñar la tabla habitacion

R02F01T01P01 – Insertar habitaciones de prueba

R02F01T02 – Crear formulario de habitaciones

R02F01T02P01 – Guardar una habitación desde la interfaz

R02F02 – Gestionar características y disponibilidad

R02F02T01 – Asociar capacidad a cada habitación

R02F02T01P01 – Verificar que la capacidad se gestiona correctamente

R02F02T02 – Preparar la lógica de disponibilidad

R02F02T02P01 – Mostrar habitaciones disponibles según fechas

R03 – El sistema debe permitir gestionar estancias con múltiples huéspedes

R03F01 – Crear estancias

R03F01T01 – Diseñar la tabla estancia

R03F01T01P01 – Insertar una estancia de prueba

R03F01T02 – Diseñar tabla intermedia estancia_huesped

R03F01T02P01 – Asociar varios huéspedes a una misma estancia

R03F01T03 – Crear interfaz para seleccionar varios huéspedes en una estancia

R03F01T03P01 – Añadir varios huéspedes y comprobar que se guardan correctamente

R03F01T04 – Asociar estancia con habitación y fechas

R03F01T04P01 – Crear una estancia completa desde la interfaz

R03F02 – Validar reglas de negocio

R03F02T01 – Validar que la fecha de salida sea posterior a la de entrada

R03F02T01P01 – Introducir fechas incorrectas y comprobar validación

R03F02T02 – Controlar disponibilidad por solapamiento de fechas

R03F02T02P01 – Intentar asignar una habitación ocupada en ese rango

R03F02T03 – Validar capacidad de la habitación respecto al número de huéspedes

R03F02T03P01 – Intentar asignar más huéspedes que la capacidad permitida

R04 – El sistema debe generar un archivo XML para la comunicación de viajeros

R04F01 – Preparar los datos necesarios para la exportación

R04F01T01 – Asegurar que los datos del huésped cumplen los requisitos (documento, nacionalidad, dirección, etc.)

R04F01T01P01 – Verificar que todos los campos obligatorios están completos

R04F01T02 – Estructurar los datos de estancia según el formato requerido

R04F01T02P01 – Comprobar que los datos se agrupan correctamente (contrato, personas, etc.)

R04F02 – Generar el archivo XML

R04F02T01 – Implementar la generación del XML siguiendo la estructura oficial

R04F02T01P01 – Generar un XML de prueba a partir de una estancia

R04F02T02 – Validar el XML generado

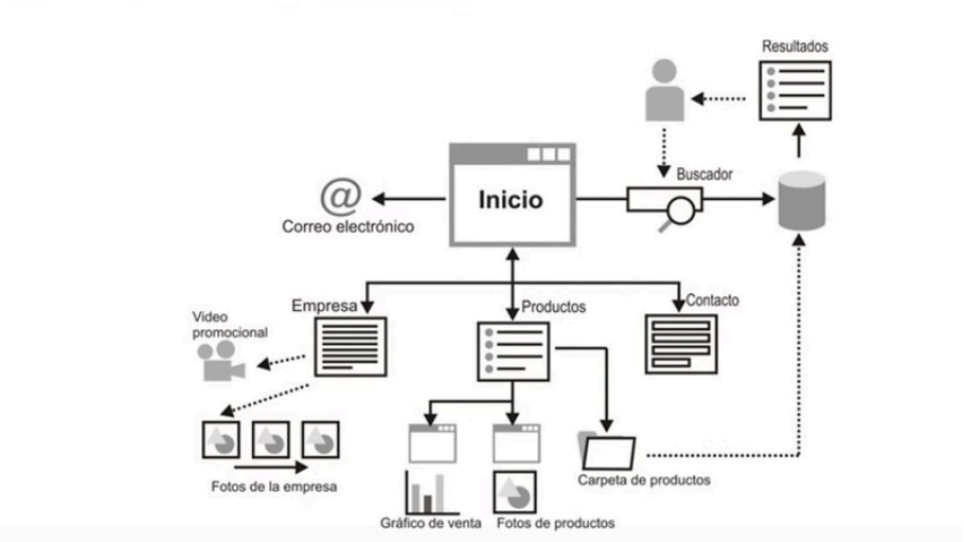
R04F02T02P01 – Comprobar que el XML cumple el formato esperado

# DESCRIPCIÓN

Se deben incluir todos los diagramas y explicaciones necesarias para entender el tipo de solución que propones en tu proyecto. Enumeramos algunos de los más comunes.
Todos deben ser perfectamente legibles.

Son ejemplos.

**Arquitectura de la solución.** Es un diagrama en el que se vea cómo funcionara el desarrollo planificado. Por ejemplo:




**Casos de uso.** Incluye diagrama y tabla con:

- Descripción.
- Precondiciones
- Postcondiciones
- Datos de entrada
- Datos de salida
- Tablas
- Clases
- Interfaces

Ejemplo:

*Caso de uso: Pedir ayuda*

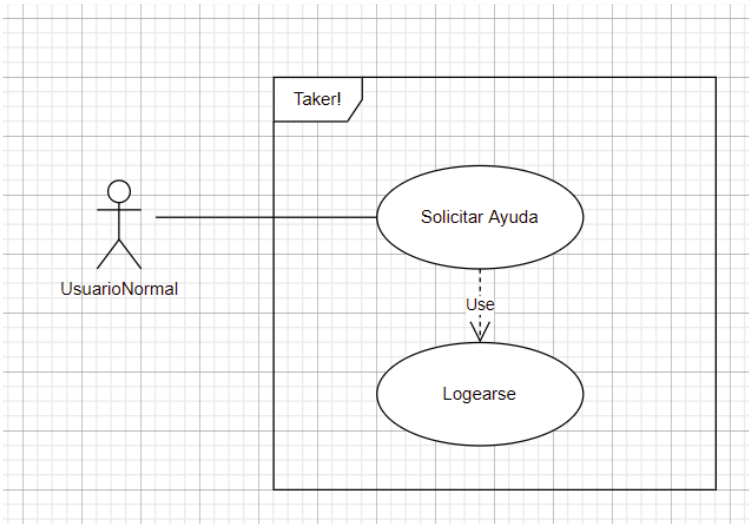



Ilustración 1: caso de uso Pedir Ayuda

| **DESCRIPCIÓN**: Solicitar ayuda al especialista | |
|---|---|
| **PRECONDICIONES**:<br><br>Usuario logado | **POSTCONDICIONES**:<br><br>Solicitud en espera<br><br>Se inicia el chat |
| **DATOS ENTRADA**<br><br>Nombre especialista<br><br>Id usuario<br><br>Id especialista | **DATOS SALIDA**<br><br>Nombre especialista<br><br>Id usuario<br><br>Id especialista<br><br>Idchat<br><br>Valoración<br><br>fecha/hora |
| **TABLAS**:<br><br>USUARIOS<br><br>CHAT | **CLASES**:<br><br>ESPECIALISTA.PHP<br><br>USUARIO NORMAL.PHP<br><br>CHAT.PHP |

| | |
|---|---|
| **INTERFACES**:<br><br>PERFILUSUARIO.HTML<br><br>CHAT.HTML | |

Tabla 1: caso de uso Pedir Ayuda

# DISEÑOS (Los que procedan según el tipo de proyecto)

**Diagrama de clases**.

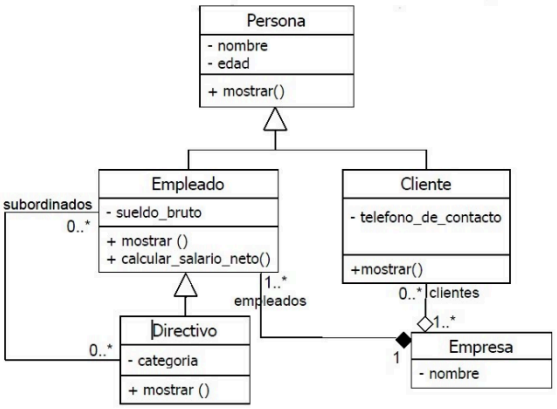



**Diagrama E/R** (Entidad - Relación)

PROMETEO

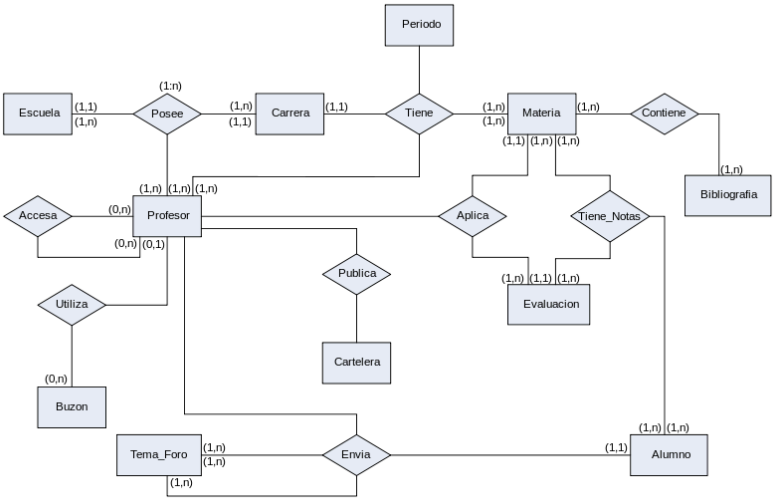



**Diagrama de la base de datos**. Con detalle de campos.

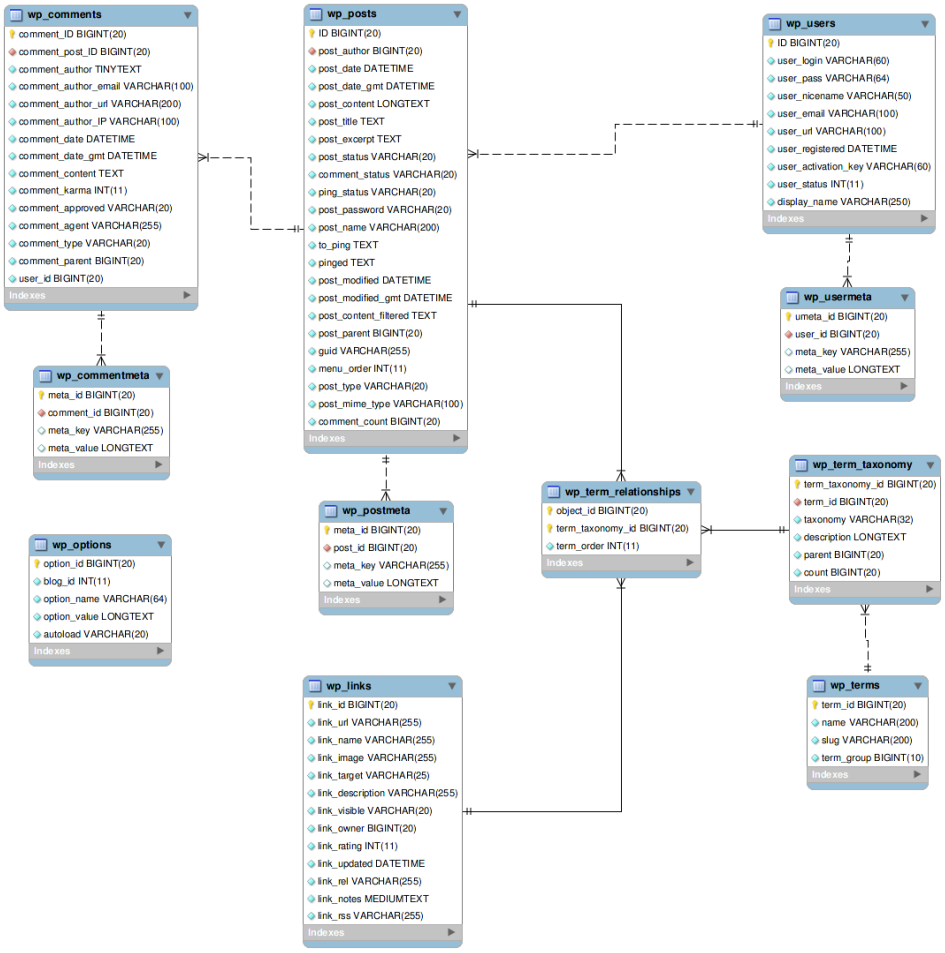
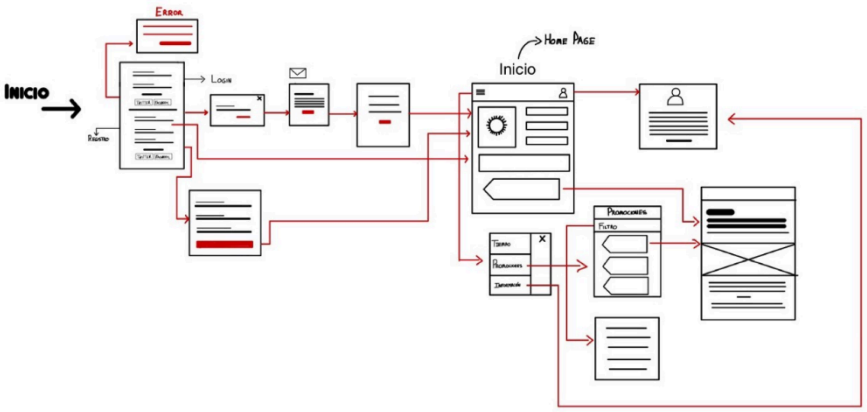
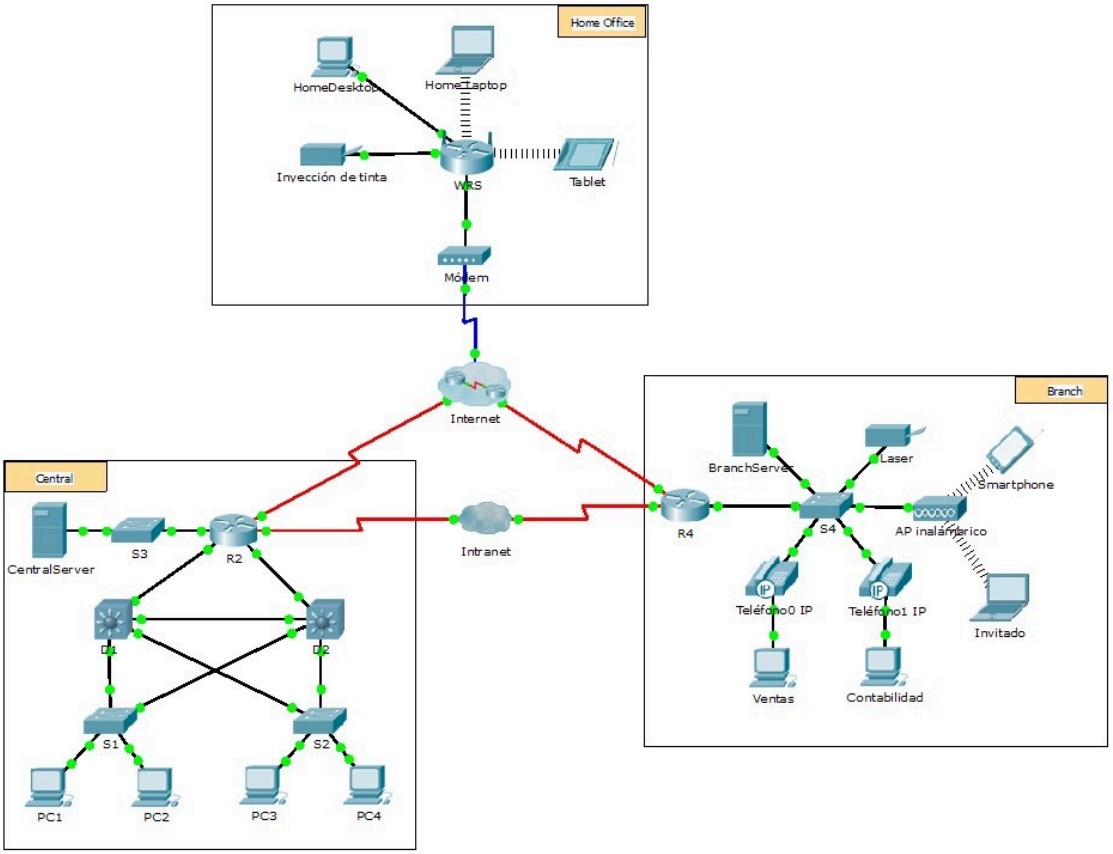



**Diagrama de flujo de navegación**. Esquemático. Debe incluirse en la propuesta.




**Interfaces**. Interesa ver la solución en diferentes tamaños o dispositivos.

**Diagrama de red.**



# TECNOLOGÍA

Las tecnologías y herramientas utilizadas para este proyecto.  Por ejemplo:

**Java**.
Descripción de la herramienta.
Descripción del uso de la herramienta en el proyecto.

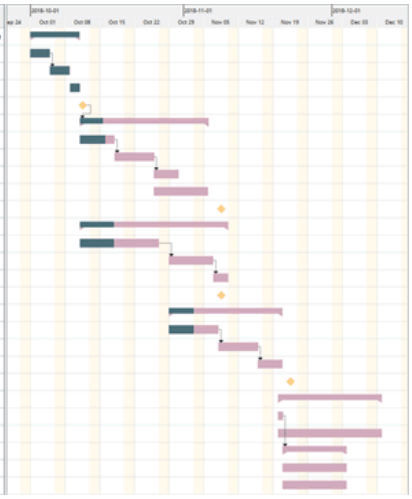
# METODOLOGÍA

**Metodología usada** y justificación de la misma.

Se presentarán dos planificaciones, una valoración inicial y previa a la implementación del proyecto y otra final con el tiempo real dedicado a cada parte del RFTP. Se analizarán las desviaciones.
El tiempo se expresará en horas. Debe existir una totalización final.

**Diagrama de Gantt** (Microsoft Project o similar). Real, contrastable con GIT, RFTP y Casos de uso.

| ID | Task | Start | Finish | Duration | Progress | Priority | Resources |
|---|---|---|---|---|---|---|---|
| 1 | **Analysis** | **2018-10-01 08:00** | **2018-10-10 16:00** | **8 d** | **98.9%** | 1 | **Rita; William; Tyler; Wenger; Steve** |
| 2 | Requirement Meetings | 2018-10-01 08:00 | 2018-10-04 16:00 | 4 d | 100% | 1 | Rita; William |
| 3 | Communication with Stakeholders | 2018-10-05 08:00 | 2018-10-08 16:00 | 2 d | 95.5% | 3 | Rita; William; Tyler; Wenger; Steve |
| 4 | Document Current System | 2018-10-09 08:00 | 2018-10-10 16:00 | 2 d | 100% | 3 | William |
| 5 | Analysis Finished | 2018-10-10 08:00 | 2018-10-10 16:00 | 1 d | 0% | 1 | |
| 6 | **Design** | **2018-10-11 08:00** | **2018-11-05 16:00** | **18 d** | **14.9%** | 2 | **Steve; Yvette; Zoe** |
| 7 | Design Database | 2018-10-11 08:00 | 2018-10-17 16:00 | 5 d | 62.4% | 1 | Steve |
| 8 | Software Design | 2018-10-18 08:00 | 2018-10-25 16:00 | 6 d | 0% | 2 | Yvette |
| 9 | Interface Design | 2018-10-26 08:00 | 2018-10-30 16:00 | 3 d | 0% | 4 | Zoe |
| 10 | Create Design Specifications | 2018-10-26 08:00 | 2018-11-05 16:00 | 7 d | 0% | 2 | Steve |
| 11 | Design Finished | 2018-11-05 08:00 | 2018-11-05 16:00 | 1 d | 0% | 2 | |
| 12 | **Development** | **2018-10-11 08:00** | **2018-11-09 16:00** | **22 d** | **22.5%** | 3 | **Tyler; Wenger** |
| 13 | Develop System Module | 2018-10-11 08:00 | 2018-10-26 16:00 | 12 d | 41.2% | 1 | Tyler; Wenger |
| 14 | Integrate System Module | 2018-10-29 08:00 | 2018-11-06 16:00 | 7 d | 0% | 1 | Tyler |
| 15 | Perform Initial Testing | 2018-11-07 08:00 | 2018-11-09 16:00 | 3 d | 0% | 2 | Wenger |
| 16 | Development Finished | 2018-11-09 08:00 | 2018-11-09 16:00 | 1 d | 0% | 3 | |
| 17 | **Testing** | **2018-10-29 08:00** | **2018-11-20 16:00** | **17 d** | **29.4%** | 4 | **Vicky; Mike** |
| 18 | Perform System Testing | 2018-10-29 08:00 | 2018-11-07 16:00 | 8 d | 62.5% | 3 | Vicky |
| 19 | Document Issues Found | 2018-11-08 08:00 | 2018-11-15 16:00 | 6 d | 0% | 4 | Mike |
| 20 | Correct Issues Found | 2018-11-16 08:00 | 2018-11-20 16:00 | 3 d | 0% | 1 | Mike |
| 21 | Testing Finished | 2018-11-20 08:00 | 2018-11-20 16:00 | 1 d | 0% | 3 | |
| 22 | **Implementation** | **2018-11-20 08:00** | **2018-12-10 16:00** | **15 d** | **0%** | 1 | **Tyler; Mike** |
| 23 | On-Site Installation | 2018-11-20 08:00 | 2018-11-20 16:00 | 1 d | 0% | 5 | Tyler |
| 24 | Support Plan for the System | 2018-11-20 08:00 | 2018-12-10 16:00 | 15 d | 0% | 4 | Mike |
| 25 | **Completion** | **2018-11-21 08:00** | **2018-12-03 16:00** | **9 d** | **0%** | 2 | **Rita; William** |
| 26 | System Maintenance | 2018-11-21 08:00 | 2018-12-03 16:00 | 9 d | 0% | 3 | Rita |
| 27 | Evaluation | 2018-11-21 08:00 | 2018-12-03 16:00 | 9 d | 0% | 1 | William |




**Presupuesto.** Con detalle de horas, indispensable si se realiza en grupo, y coste total del desarrollo por cada requisito.

**README y GIT.**

# TRABAJOS FUTUROS

Trabajos de ampliación y mejora proyectados.

# CONCLUSIONES

Conclusión profesional del proyecto.

# REFERENCIAS

Según las normas APA.

Cada referencia se acompañará de un texto descriptivo con el apartado del proyecto asociado.

**Formato:**

Autor, A. A. (Año de publicación). Título de la página. Recuperado de URL

**Ejemplo:**

*Aplicado en la investigación del tema de la web.*

Smith, J. (2023). La importancia del reciclaje en la conservación del medio ambiente. Recuperado de https://www.ejemplodepagina.com/

**Otro ejemplo:**

*Aplicado para realizar las vistas de la base de datos.*

Oracle Corporation. (s. f.). Oracle Database 19c Documentation. Recuperado de https://docs.oracle.com/en/database/oracle/oracle-database/index.html